Trivet: Proposal, version 2

App name

Trivet

The problem, in one sentence

A second-year student juggling coursework, a part-time job, the gym and a reading habit has no single place that shows how many hours each of those took this week, and whether one is crowding out the others.

Who is this for

- **Who:** university students and early-career workers aged 18-30 whose week is self-directed rather than fixed by an employer, timetable or parent.
- **What they do today:** a to-do app or notes for work, a separate workout app (or nothing) for exercise, and Goodreads or memory for books and shows. None of these compare time across areas.

Core features (MVP), revised

#	Feature	Still in the MVP?	Flutter pieces it needs	Honest estimate
1	Work: add project, log dated hours, set status	keep	`ListView.builder`, `Card`, `TextField`, `DropdownButton`, `showDialog` (log hours), `Project` and `WorkLog` models	7 h
2	Health: log workout, weekly streak	keep	`ListView.builder`, `TextField`, `showDatePicker`, `Workout` model, streak computed from dates	5 h
3	Leisure: log media with status and rating, log time spent	keep	`ListView.builder`, `SegmentedButton` (want / in progress / done), row of star `Icons`, `MediaEntry` and `LeisureLog` models	6 h
4	Dashboard: hours per pillar this week side by side, plus nudge text	keep	`Row`/`Column` of three `Cards`, `Text` nudges, a week-range filter over the logs	5 h
5	Add Entry shared modal	keep	`showModalBottomSheet`, `SegmentedButton` to pick the pillar, `Form` with `TextFormField` validators	4 h
6	Saving and loading	keep	one store class per model, `load()` in `initState`, `save()` after `setState`, loading indicator	3 h
7	Radar / triad chart on the Dashboard	stretch	`CustomPainter` on a `CustomPaint` widget	4 h
8	Routine builder inside the Health screen	stretch	`Routine` model, nested list editing, `ReorderableListView`	6 h

MVP total: about 30 hours; with both stretch items, about 40 hours. I have about 3-4 weeks left in the term, so this is roughly 8-10 hours a week for the MVP. m5a5 took me about 4-5 hours for 2-3

screens, which is roughly 2 hours per screen of plain UI. My feature estimates are higher than that because each feature here also needs a model class, saving and loading, and a week filter, none of which m4a4 and m5a5 required. I will re-score these numbers after the storage spike on September 24.

Stretch goals

1. Radar / triad chart of the week's hours, drawn with `CustomPainter` on the Dashboard
2. Routine builder (reusable workout templates) inside the Health screen

How my app saves data

- **If two different people install my app, should they see the same data?** No. A person's work, workouts and reading are personal. There are no accounts, no sharing and no feature where one user reads another's data.
- **Roughly how many records in a realistic week:** about 30-50 small records. Over a full term that is a few hundred.
- **My choice:** `shared_preferences`, with one key per model: `projects`, `worklogs`, `workouts`, `media` and `leisurelogs` for the MVP, plus `routines` if I build the stretch routine builder. Each key holds a JSON-encoded `List<String>`, one JSON string per record, using `toMap` / `fromMap` and `dart:convert`.
- **Why this one and not the others:** The app is single-user and fully offline, so Firebase or Supabase would cost a day of setup and give me no feature in return. Hive would also work and runs on web, but it needs type adapters and a box per model, and its main benefit is handling large record counts, which I will not reach: a few hundred small records is well within what `shared_preferences` handles. Drift adds code generation and a schema for relationships I do not have. The unit itself names a tracker as the `shared_preferences` case. It takes about 20 minutes, runs in a Codespace and on web, and leaves the time for the dashboard. **Tradeoff accepted:** every change rewrites the whole list for that key, and there are no queries, so filtering by "this week" happens in Dart after loading everything. That is fine at this size. If the history grows into the thousands, I would move to Hive.
- **What I save, concretely:**

Key	Class	Fields
`projects`	Project	id, title, status
`worklogs`	WorkLog	id, projectId, hours, date
`workouts`	Workout	id, type, durationMinutes, date, notes
`media`	MediaEntry	id, title, type, status, rating
`leisurelogs`	LeisureLog	id, mediaId, minutes, date
`routines` (stretch, only if I build the routine builder)	Routine	id, name, exercises

- **Have I tried it yet?** Not yet. I will do the one-hour spike on a throwaway branch starting September 24.

One thing I want to add that the course did not teach

A radar / triad chart of the week's hours per pillar, drawn with `CustomPainter`, so it needs no package.

- **Package:** none. `CustomPaint` is part of Flutter.
- **Runs where I develop:** yes, it is plain Flutter drawing, so it runs in a browser and in a Codespace with no fallback needed.
- **Core or stretch: stretch.** The app works without it because the Dashboard already shows the three hour totals side by side. I will spike the smallest version (a triangle with three points) for one hour before committing to it.

How my project runs when someone else opens it

- I am keeping the `device_preview` wrapper: **yes**, with `enabled: !kReleaseMode`, as page 13 says.
- My app runs in a browser with `flutter run -d web-server`, start to finish, with every screen reachable: **yes.** I will re-test every screen on a clean checkout before submitting.
- Anything that needs real hardware degrades to sample data instead of crashing: **not applicable.** Trivet uses no camera, GPS, sensors or notifications.
- Public repository, in my own GitHub account: **no API keys, passwords or real personal data are needed.** Storage is local, so there is no backend key. All seed data is invented and no real names or personal information will appear in the repo, screenshots or video.

Data the app remembers

Thing	Fields	Where it is saved
Project	id, title, status	`shared_preferences` key `projects`
WorkLog	id, projectId, hours, date	`shared_preferences` key `worklogs`
Workout	id, type, durationMinutes, date, notes	`shared_preferences` key `workouts`
MediaEntry	id, title, type, status, rating	`shared_preferences` key `media`
LeisureLog	id, mediaId, minutes, date	`shared_preferences` key `leisurelogs`
Routine (stretch)	id, name, exercises	`shared_preferences` key `routines`
Weekly hours per pillar, streak, nudges	(derived)	not saved; computed from the logs

Screens

1. Dashboard (home)
2. Work
3. Health
4. Leisure
5. Add Entry (shared bottom sheet)

Risks, revised

- **The risk I named last time (the Balance Score being arbitrary):** it is gone. Hours, workouts and leisure sessions have no natural conversion rate, so any single score would have been a guess. I removed the score and Trivet now shows hours per pillar side by side, so there is no made-up conversion to defend. What could still go wrong is that three hour totals side by side are not enough to show whether a week is balanced. First step: write out one full example week of hours per pillar and check that the three totals read clearly on their own, without a score. **Date: September 24.**
- **A new risk I did not see before: weekly hours need dated entries and correct week logic.** The radar and dashboard need hours per pillar for a given week, which my prelim's single running `hoursLogged` total could not give. I changed the models to save dated logs, but filtering three kinds of logs by "this week" and computing the streak across week boundaries is easy to get subtly wrong. First step: write one week-range function and test it with `flutter test` on hand-picked dates (Sunday night, Monday morning, a gap week) before building the Dashboard. **Date: September 28.**

What changed, and why

Section	Prelim said	Now says	Why it changed
Core features	Balance Score in the MVP	Balance Score removed	The app is about hours spent in each area, not a score, and the score was the least defensible part of the plan
Core features	Radar chart and routine builder in the MVP	Both moved to stretch, inside existing screens	m5a5 took me about 4-5 hours for 2-3 screens of plain UI, and my features also need models, saving and week filtering, so about 40 hours with both stretch items does not safely fit in 3-4 weeks. I am committing to about 30 hours of MVP first and building the stretch items only if time is left
Data model	one running `hoursLogged` on Project	dated `WorkLog` and `LeisureLog` entries	A radar or weekly view of hours cannot be built from a single running total, so each log needs a date
Data model	`clientModeEnabled` field on Project	Removed	No screen or feature uses it, so the data table and screens would not agree
Persistence	"one Hive box per model"	`shared_preferences`, one key per model	Single user, offline, a few hundred records over a term; the unit names a tracker as the `shared_preferences` case and it saves setup time for the Dashboard
Risk	Arbitrary Balance Score	Score risk gone; new risk on week-range logic, first step September 30	Removing the score removed the old risk, and dated logs created the new one